WIDS ENDTERM REPORT

https://github.com/Chirayu-Jain2/WIDS-2025-Intro-to-Reinforcement-LearningI'm Chirayu Jain, a first year student in the CSE dept. I had heard about RL before WIDS from a few YouTube videos which vaguely explained how RL works on reward based learning, the idea seemed interesting to me and I wanted to learn more about exactly the math it works. Hence I took this WIDS and enjoyed learning about RL and finally solving the 15 puzzle game via RL.

Reinforcement learning is a learning framework in which an agent interacts with an environment by taking actions, receives scalar rewards, and learns a policy that maximizes expected cumulative reward over time through trial and error, without being told the correct actions.

For this WIDS, we followed the books Sutton & Barto: Reinforcement Learning & Grokking: Deep Reinforcement Learning.

WEEK 0

This week was focussed only on learning about tools like Matplotlib and NumPy which we would be using throughout the project for the next 4 weeks.

We learned python syntax as it was the language we were supposed to code in.

We learned using the NumPy library -

1. Learned the basics of NumPy and why it is important for fast numerical and scientific computing in Python.

2. Understood how to create and work with NumPy arrays, including their shape, data types, slicing, and reshaping.

3. Performed mathematical and statistical operations such as sum, mean, standard deviation, and element-wise calculations.

4. Learned matrix and vector operations including dot product, matrix multiplication, transpose, determinant, inverse, and eigenvalues.

5. Understood broadcasting and boolean indexing, which allow operations on arrays of different shapes and conditional data selection.

6. Generated random numbers with seeds for reproducibility and learned how to save and load arrays using NumPy file operations.

We learned using the Matplotlib library -

1. Learned the basics of Matplotlib and how to create visualizations using matplotlib.pyplot

2. Created and customized line plots, including markers, colors, line styles, labels, titles, legends, and gridlines.

3. Worked with subplots and layouts, understanding figures vs axes, shared axes, constrained layouts, and complex subplot arrangements.

4. Visualized data using scatter plots, bar plots (vertical, horizontal, grouped, stacked), and stack plots, with extensive customization.

5. Learned advanced visualizations such as heatmaps, contour plots, filled contours, and image-based plots.

6. Added annotations, text, equations, arrows, and labels to make plots more informative and readable.

7. Used statistical plots like box plots, violin plots, histograms, and compared different histogram styles.

8. Explored 3D plotting including scatter plots, bar charts, surface plots, wireframes, and contour projections.

9. Learned how to save and export figures with custom resolution, background, and layout options.

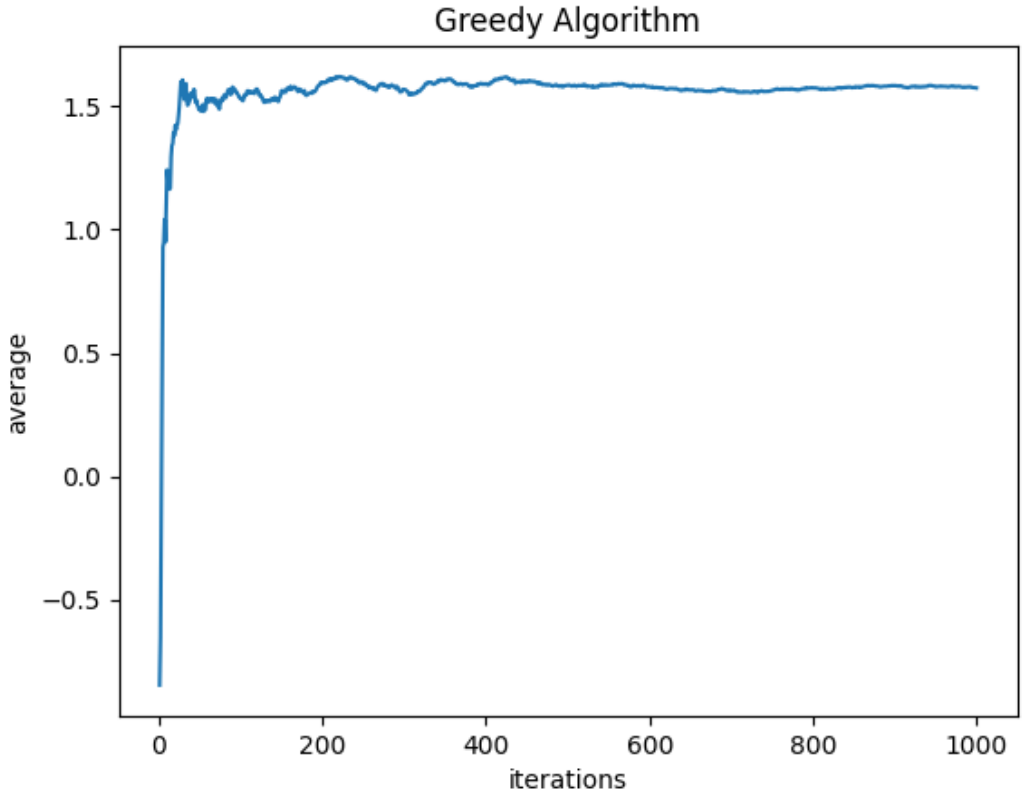
Week 1

This week was focused on understanding value function and policies, and implementing algorithms on the Multi-Armed Bandit Problem
Policy:- A policy π is a mapping from states to a probability distribution over actions:
Value function:- The value function of a policy π is the expected return starting from state s and thereafter following π

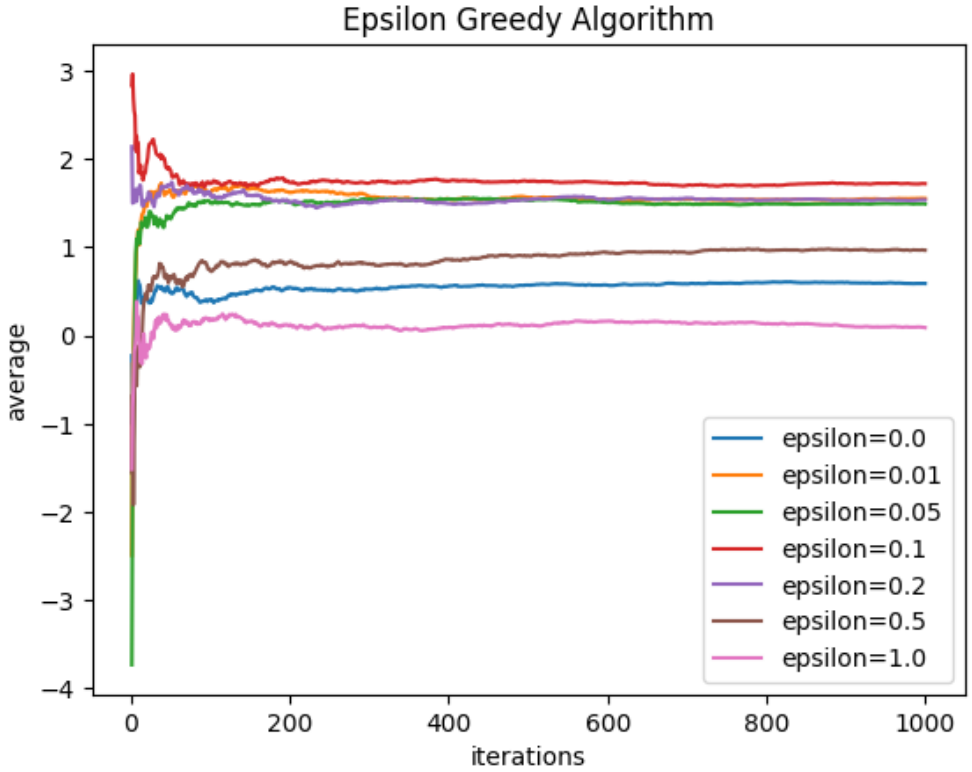
The multi-armed bandit problem models an agent facing a set of k actions (arms), each producing stochastic rewards from a Normal distribution with unknown mean. At each time step, the agent chooses one arm and observes its reward, with the objective of maximizing cumulative reward over time. In our version, there are 10 arms.

We implemented the following algorithms

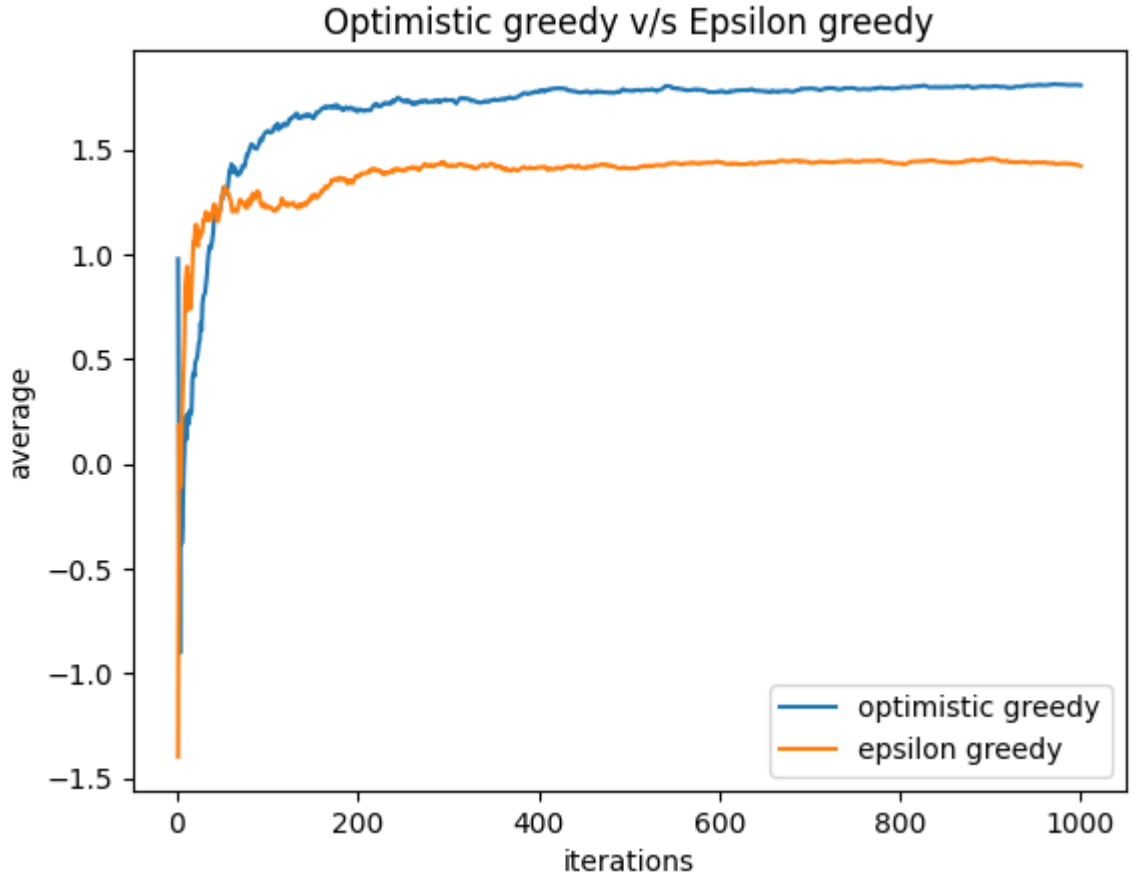
1.) Greedy Algorithm:- Always selects the action with the highest estimated value so far. It exploits current knowledge but does no exploration



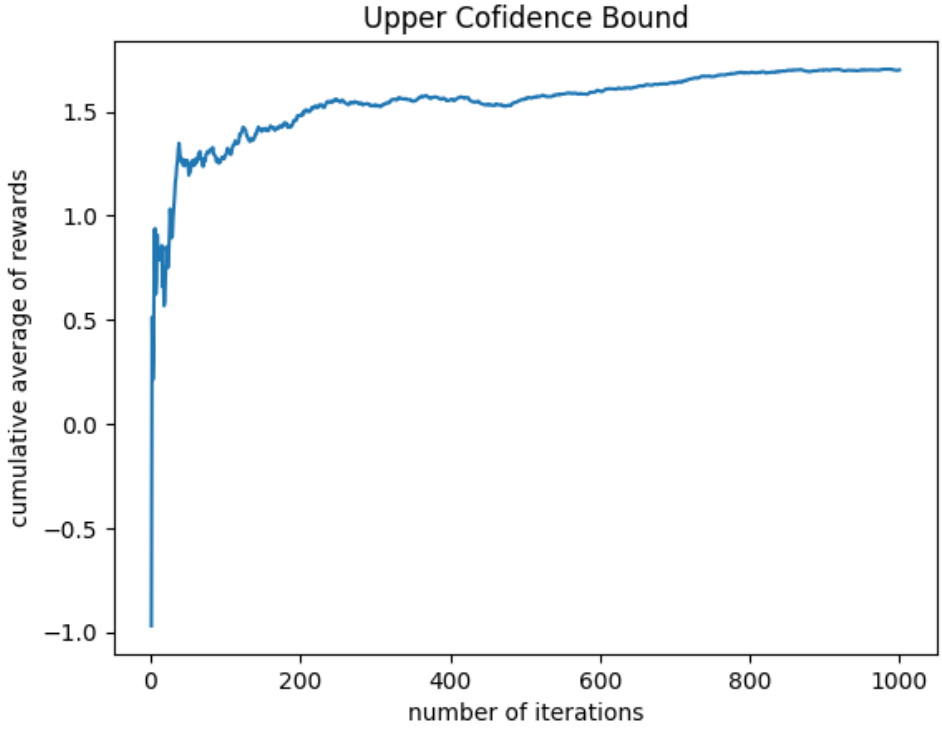
2.) Epsilon Greedy Algorithm:- With probability 1−ε, chooses the greedy (best-known) action, and with probability ε, selects a random action. This has both exploration and exploitation



3.) Optimistic Initial Values:- Initialize all action-value estimates to unrealistically high values. Early actions tend to look bad after being tried once, which naturally encourages exploration without explicit randomness



4.) Upper Confidence Bound:- Selects actions based on both estimated value and uncertainty, choosing the action that maximizes Q(a) + c * root(ln(t) / N(a))



Week 2

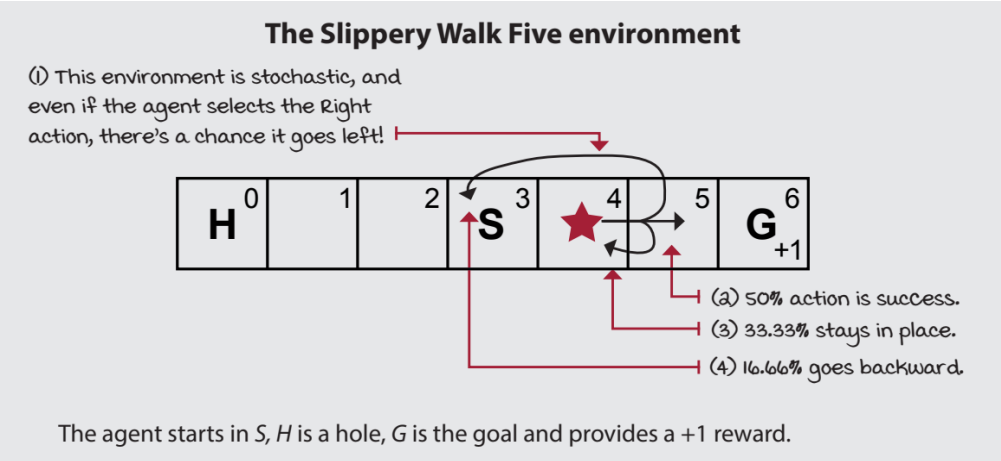
This week consisted of understanding Markov decision processes and coding them and also understanding Bellman Optimality Equations.

A Markov Decision Process (MDP) is a mathematical framework used to model sequential decision-making where outcomes are partly random and partly controlled by an agent. MDPs define an agent interacting with an environment over discrete time steps to maximize long-term rewards, adhering to the Markov property where future states depend only on the current state and action.An MDP consists of the 5 parts

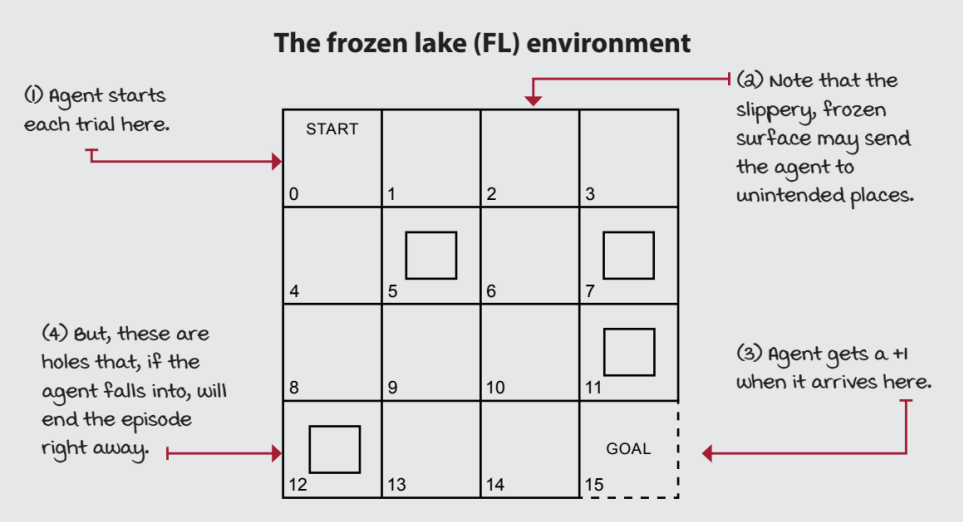
1.) S = Set of all possible states
2.) A = Set of all possible actions
3.) P = transition probability function
4.) R = reward function
5.) Gamma = discount factor

After this, we coded in various sample MDPs, which were

1.) Slippery Walk

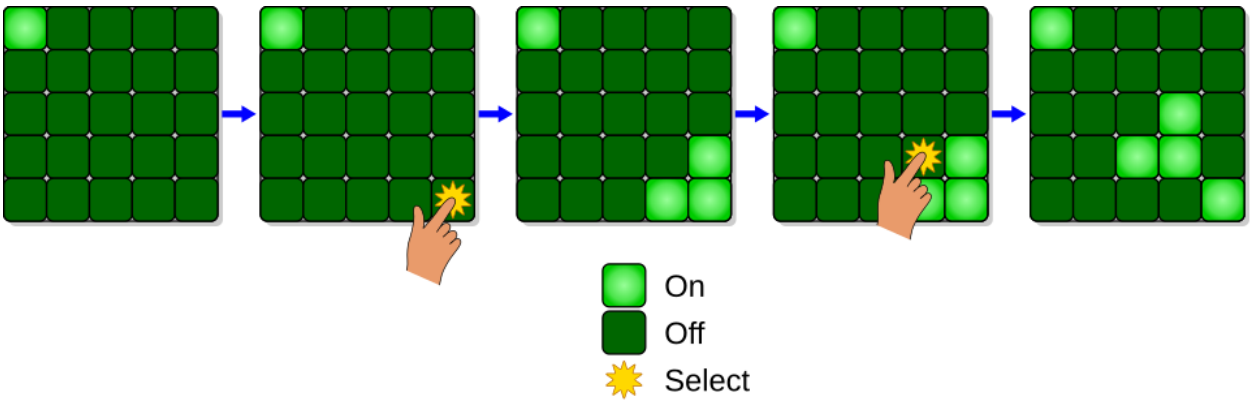


2.) Frozen Lake



3.) A deterministic version of the frozen Lake, but with a lock on the goal state and a key, and so slightly different state functions.

We also made an MDP for a game called Lights Out



Lights Out is an electronic game released by Tiger Electronics in 1995 The game consists of a 5 by 5 grid of lights. When the game starts, a random number or a stored pattern of these lights is switched on. Pressing any of the lights will toggle it and the adjacent lights. The goal of the puzzle is to switch all the lights off, preferably with as few button presses as possible.

We created a 4x4 version of Lights Out with the following attributes

- The states should represent the game grid layout
- The actions are the cells you wish to toggle
- Give a reward of 1 when solved
- The ONLY terminal state is the fully solved state

We also studied Bellman optimality equations, which allow us to iteratively compute the optimal value functions.

Bellman Equation for value functions,

Bellman Equation for v_π:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) \sum_{\substack{s' \in \mathcal{S} \\ r \in \mathcal{R}}} p(s', r|s, a)[r + \gamma v_\pi(s')]$$

Bellman Equation for action value functions,

Bellman Equation for q_π:

$$q_\pi(s, a) = \sum_{\substack{s' \in \mathcal{S} \\ r \in \mathcal{R}}} p(s', r|s, a)\left[r + \gamma \sum_{a' \in \mathcal{A}(s')} \pi(a'|s') q_\pi(s', a')\right]$$

Week 3

This week consisted of learning about policy iteration and Value iteration and solving two problems from Sutton & Barto and solving the Lights out game made previously,
The first problem we had to solve was Jack's Car rentals which is formulated as

1.) Jack manages two locations for a nationwide car rental company. Each day, some number of customers arrive at each location to rent cars. If Jack has a car available, he rents it out and is credited $10 by the national company. If he is out of cars at that location, then the business is lost. Cars become available for renting the day after they are returned. To help ensure that cars are available where they are needed, Jack can move them between the two locations overnight, at a cost of $2 per car moved. We assume that the number of cars requested and returned at each location are Poisson random variable, meaning that the probability that the number is n is n n! e, where is the expected number. Suppose is 3 and 4 for rental requests at the first and second locations and 3 and 2 for returns. To simplify the problem slightly, we assume that there can be no more than 20 cars at each location (any additional cars are returned to the nationwide company, and thus disappear from the problem) and a maximum of five cars can be moved from one location to the other in one night.

Now, this we solved using policy iteration, which is a combination of policy evaluation, where you use the policy to make a better value function, and policy improvement, where you use the value function to generate a better policy

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

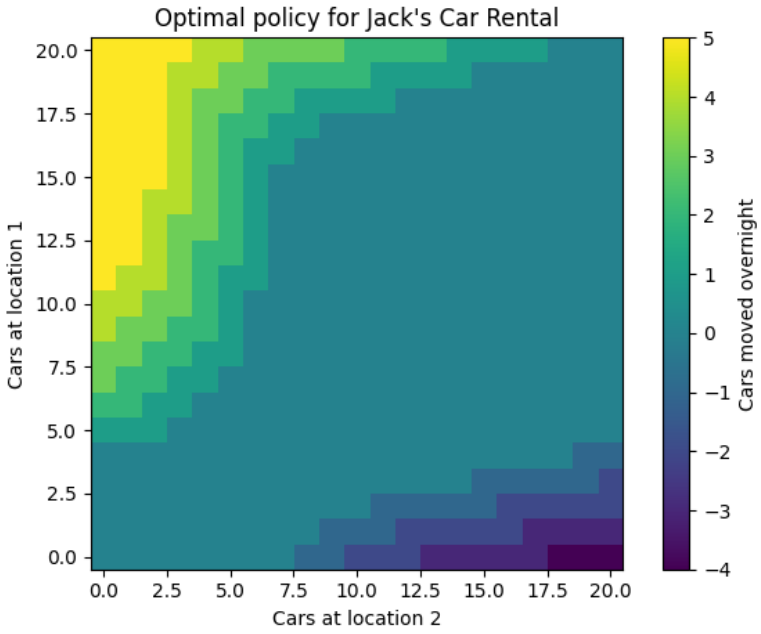
1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\left[r + \gamma V(s')\right]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

.

My solution to this looked like



The second problem we had to solve was the Gambler's problem which is formulated as

2.) A gambler has the opportunity to make bets on the outcomes of a sequence of coin flips. If the coin comes up heads, he wins as many dollars as he has staked on that flip; if it is tails, he loses his stake. The game ends when the gambler wins by reaching his goal of $100, or loses by running out of money. On each flip, the gambler must decide what portion of his capital to stake, in integer numbers of dollars.
For this, we had to implement Value iteration, which involves repeatedly generating more accurate value functions. Until the value function converges and then using that value function to create a policy.

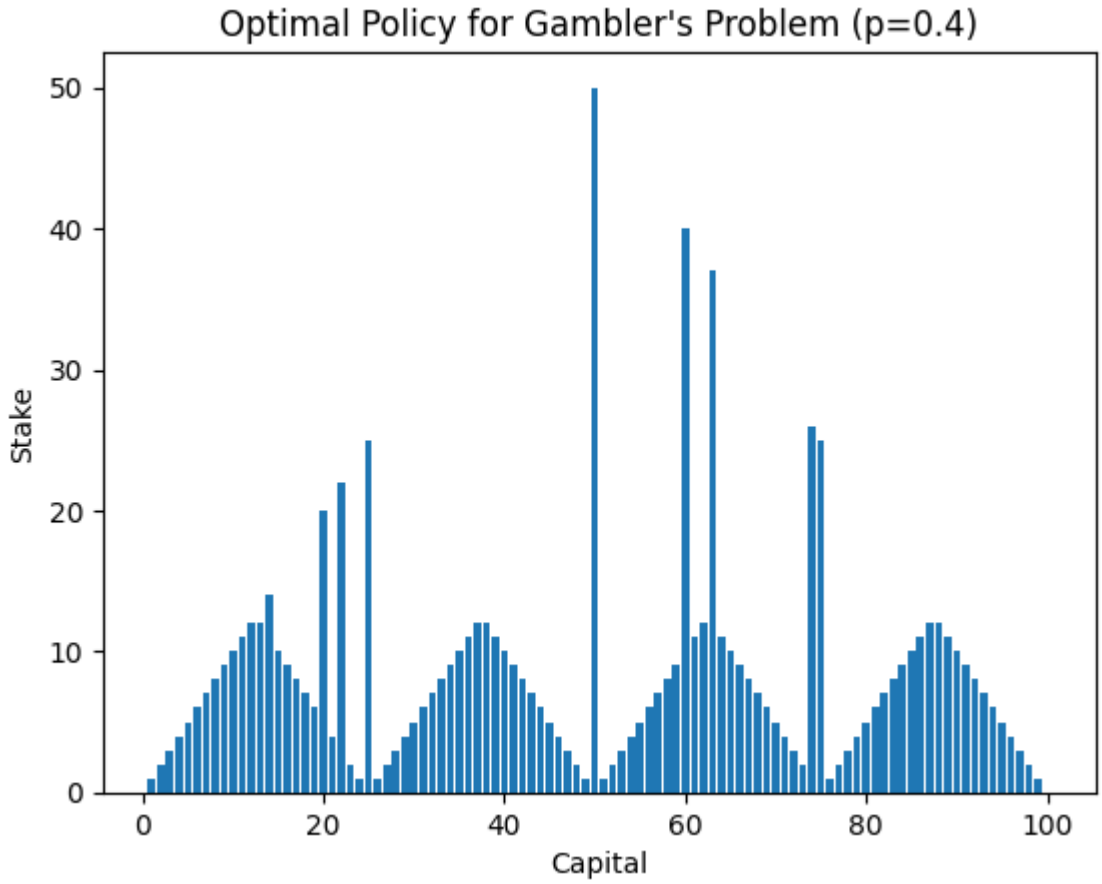
Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $v \leftarrow V(s)$
| $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$
| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

So the solution I made looks like



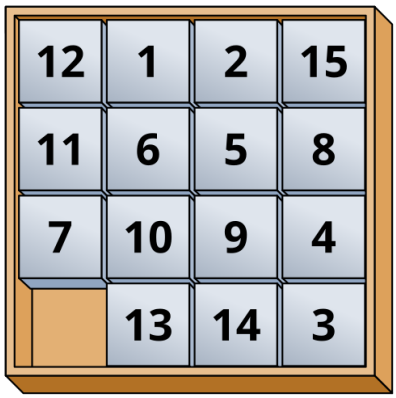
The graph shows that the optimal policy is not smooth and has sudden jumps in the stake at certain capital values. Instead of increasing gradually, the policy often recommends large or even all-in bets, especially when a big jump can help reach the target capital faster. The repeated triangular patterns indicate that the goal is to reach the final capital exactly, not to play safely at every step. Since the win probability is p = 0.4, the gambler is at a disadvantage, so the optimal strategy favors bold, risk-taking bets over cautious ones.

Then we used the MDP from the previous week and used it to try to find the optimal way to solve Lights Out

While solving the Lights Out MDP, it became clear that the number of states increases very fast, which makes the problem slow to solve even for a small 4×4 grid. Even though the transitions are deterministic, pressing one button changes multiple lights, so it's hard to predict the effect of a single action. Value iteration took a long time to converge because many states look similar and keep updating by small amounts. Overall, the problem showed how a simple-looking puzzle can become much more complex when modeled as an MDP, especially in terms of computation and state representation.

Week 4

For the final week, we made a RL based solver for the 15 puzzle.
The **15 puzzle** is a sliding puzzle. It has 15 square tiles numbered 1 to 15 in a frame that is 4 tile positions high and 4 tile positions wide, with one unoccupied position. Tiles in the same row or column of the open position can be moved by sliding them horizontally or vertically, respectively. The goal of the puzzle is to place the tiles in numerical order (from left to right, top to bottom).



To solve it we followed the ideas presented in the article https://medium.com/@amshali/15-puzzle-with-reinforcement-learning-8bcfc1aa54e7
to break the '15 puzzle' into 3 different parts and solve them individually.
The three parts are

1.) Solve the first row
2.) Solve the second row
3.) Solve the rest

I wrote the code in Python, and using it I managed to solve the '15 puzzle' in around ~60-70 moves.

Here's the link to my github repo for WIDS -
https://github.com/Chirayu-Jain2/WIDS-2025-Intro-to-Reinforcement-Learning